PRIORITY INVERSION IN REAL-TIME OPERATING SYSTEMS

ADVANCES IN SECURE SCHEDULING FOR REAL-TIME ENVIRONMENTS

Joseph Rissman

CS575 Operating Systems

April 29th, 2025

Video Presentation: https://youtu.be/JGlVL1sQQoE

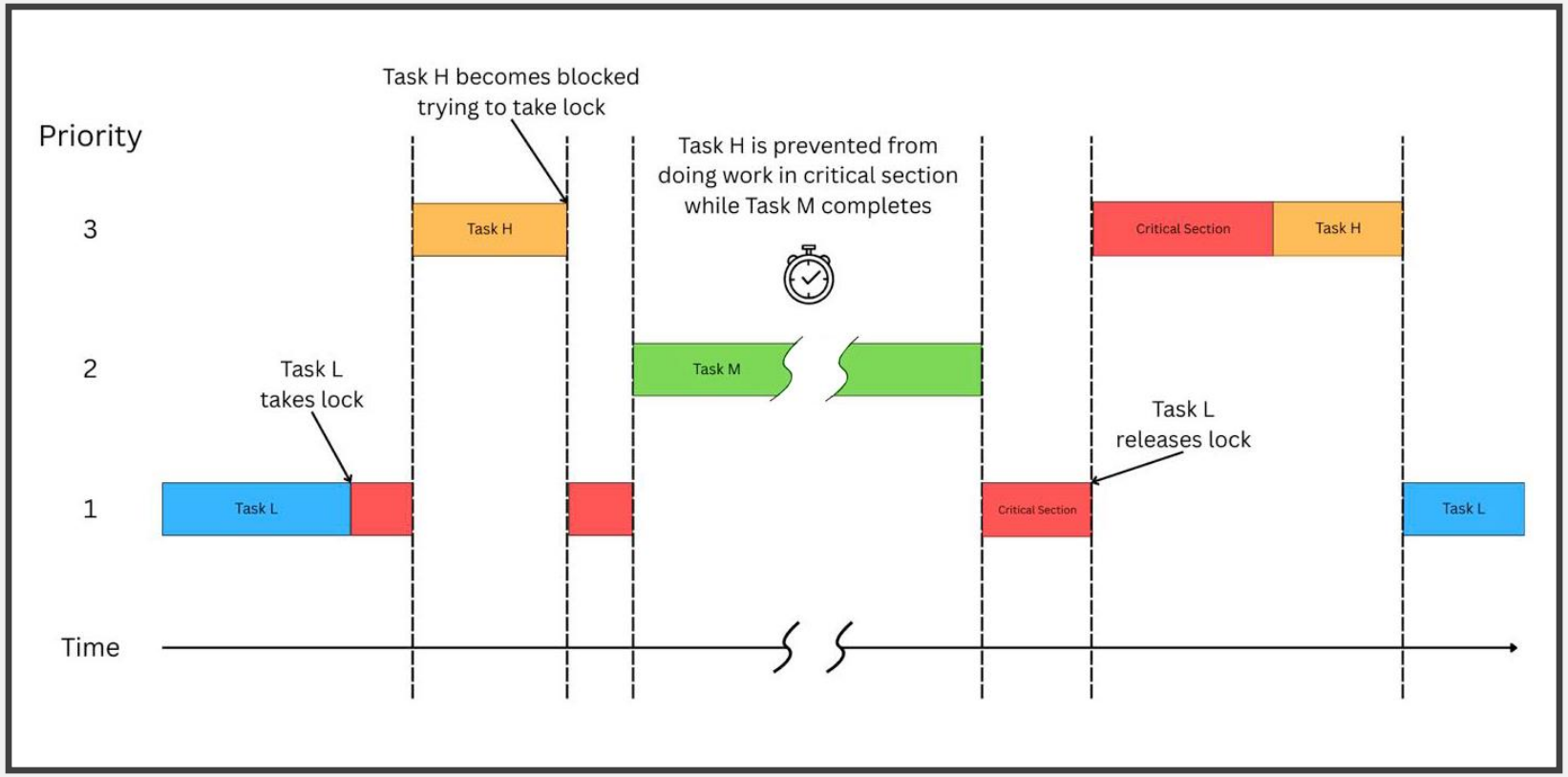
Understanding Real Time Systems and Priority Inversion

- Real time systems require tasks to complete by strict deadlines for correct operation.
- Common scheduling methods like RMS and DDS use task priorities to manage execution.
- Priority inversion is a problem where a high priority task is delayed by a lower priority task.
- Often occurs when tasks compete for shared resources managed using synchronization tools like semaphores or locks.
- These tools can cause blocking where a high priority task must wait for a lower priority task to release a resource.

The Risks and Impacts of Unbounded Priority Inversion

- Unbounded priority inversion occurs when blocking duration is unpredictable.
- A high priority task can be blocked by intermediate priority tasks that do not share resource resources, which extends the high priority task's waiting time unexpectedly.
- This uncontrolled blocking seriously harms system performance and predictability.
- High priority tasks might miss critical deadlines even if the system is not busy.

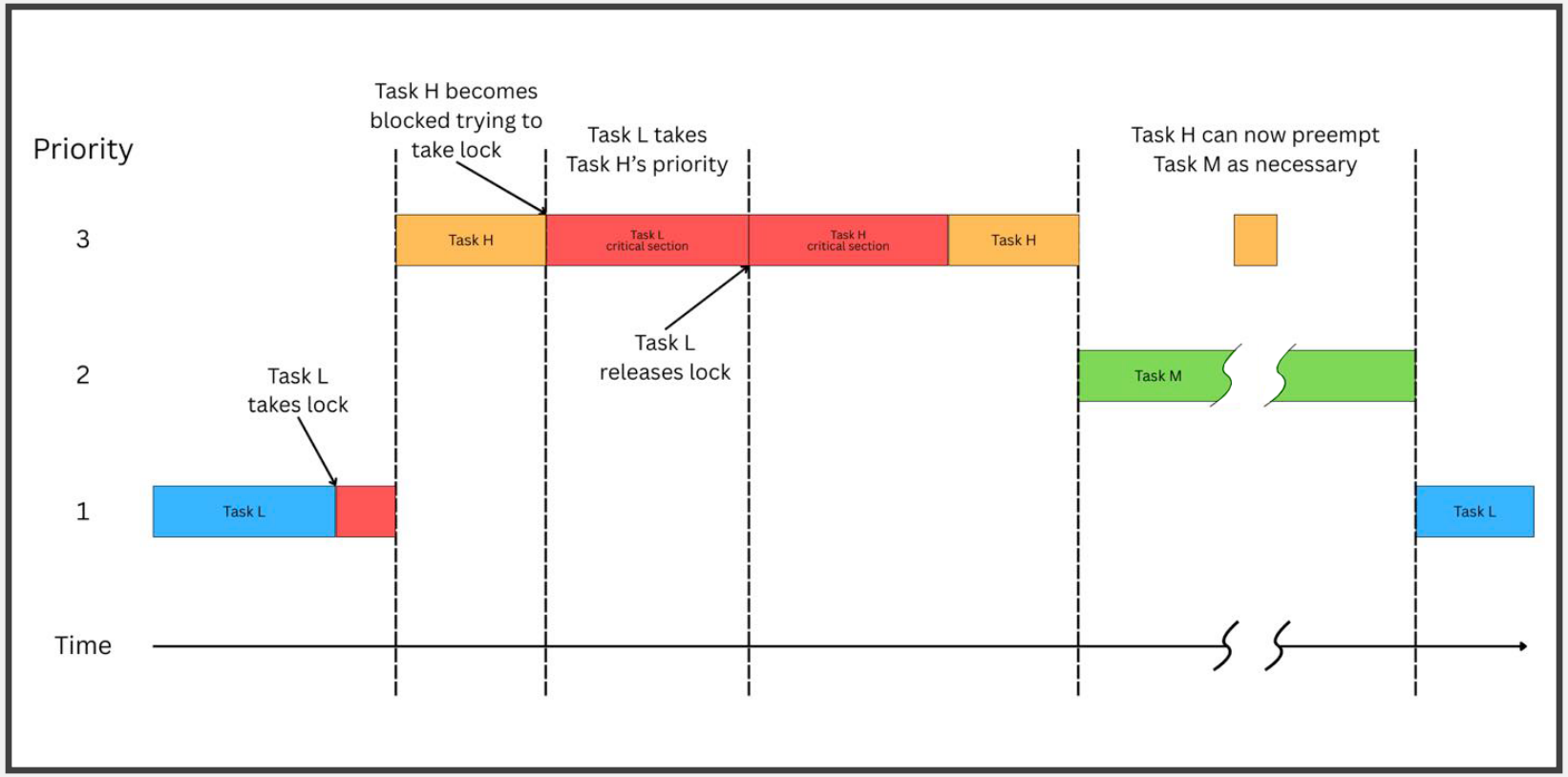
Unbounded Priority Inversion



Foundational Techniques to Address Priority Inversion in Real-time Systems

- Basic protocols aim to minimize and bound priority inversion blocking times.
- The Priority Inheritance Protocol or PIP is one common approach.
- With PIP, a lower priority task holding a needed resource temporarily inherits the higher priority of the task it is blocking.
- This inheritance prevents medium priority tasks from delaying the release of the resource.
- Other related methods like the Priority Ceiling Protocol also help manage resource contention but are not explored further here.

Priority Inheritance Protocol (PIP)



Timing Channels and Side Channel Attack Vulnerabilities

- Predictable scheduling in real time systems creates security risks.
- Attackers exploit timing patterns to infer sensitive system information or disrupt tasks.
- Timing side channels allow attackers to learn about task schedules by observing execution intervals or preemptions.
- Priority inversion events and resource contention contribute to the timing variations that these attacks exploit.
- Covert timing channels represent another way hidden information can be communicated between system components.

Mitigation via Randomization for Covert Channels Using TimeDice

- TimeDice targets covert timing channels between system partitions.
- It introduces randomness into the partition scheduling process.
- At runtime it dynamically and randomly inverts the priorities of partitions.
- This randomization includes checks to ensure real time deadlines are still met.
- The goal is to disrupt timing patterns making covert channels less effective.

Mitigation via Enhanced Randomization for Side Channels Using REORDER++

- REORDER++ combats side channel attacks through enhanced schedule randomization.
- It uses controlled random priority inversions to disrupt predictable task execution patterns.
- A key feature is its online test using runtime data to safely maximize randomization opportunities.
- This improves security against timing inference attacks without harming system schedulability.

Mitigation by Preventing Leakage via Shared Resources Using FT-PIP

- FT-PIP focuses on preventing information leakage through shared resources.
- The protocol combines Priority Inheritance Protocol PIP with a Flush Task (FT) mechanism.
- Flush tasks are used conditionally to sanitize resource states like caches.
- This prevents data remnants from being exploited with side channel attacks while managing priority inversion.

Comparing Security Approaches Randomization and Resource Sanitization

- The discussed protocols represent two main security strategies – randomization and resource sanitization.
- Randomization used by TimeDice and REORDER++ fights timing attacks by making schedules unpredictable, while maintaining schedulability and ensuring real-time deadlines are met.
- Resource sanitization used by FT-PIP prevents data leakage directly via shared components like caches.
- These methods address different vulnerability types using priority inversion, ultimately improving overall system security.

Balancing Security with Real Time Guarantees

- Recent protocols offer distinct strategies for enhancing real time system security.
- Methods like TimeDice REORDER++ and FT-PIP target specific threats including timing channels and resource data leakage.
- These advanced scheduling techniques represent ongoing efforts to create more robust and resilient systems.
- A key challenge remains successfully balancing strict timing requirements with effective defenses against sophisticated attacks.